```
%load_ext tutormagic

from IPython.core.display import HTML

HTML("""
<style>
ul {
    margin-top: 0.3em !important;
    margin-bottom: 0.3em !important;+
    padding-left: 1em !important; /* Einzug anpassen */S
}
</style>
""")
```

Programmierung für KI

Wintersemester 2026/27

Prof. Dr. Heiner Giefers

Python Basics

Bezeichner und Ausdrücke

- Variablen
- Formatierung und Kommentare
- Ausdrücke
- Boolsche Ausdrücke

- Bedingte und alternative Ausführung
- Schleifen (`while` und `for`)
- Funktionen

- Dateien, Zeichenketten, ...

- In Python gibt es keine (klassischen) Variablen
- Objekte können mit einem *Namen* bezeichnet werden
- Objekte haben einen Typ

```
a = 12
b = 12.0
c = "Hallo Welt!"
d = "Hallo" + " Welt!"
```

```
%%tutor --lang python3 --heapPrimitives --secure
a = 12
b = 12.0
c = "Hallo Welt!"
d = "Hallo" + " Welt!"
```

Bezeichner

- Klein- oder Großbuchstaben, Ziffern, oder Unterstriche (_)
- Das erste Zeichen
  - darf keine Ziffer sein
  - sollte kein Unterstrich sein (Namen, die mit Unterstrich(en) beginnen, haben eine spezielle Bedeutung)
- Schlüsselwörter können nicht als Bezeichner gewählt werden
- Funktionen und viele Konstanten sind keine Schlüsselwörter
- Wenn man nicht aufpasst, kann man Namen überdecken (”umdefinieren”)

```
#and = 5
#and + and
xxx = print
xxx("Hello")
```

Es kann auch sinnvoll sein, Standard-Funktionen umzubenennen

```
def myinput(s):
    print(s)
    return 42

i_old = input
input = myinput
x = input("Gib eine Zahl ein: ")
print(x)
input = i_old
```

Die `print` Anweisung

```
laenge = 5
print("Die Kantenlänge ist", laenge, "cm")
print("Die Kantenlänge ist %d cm" % (laenge))
print(f"Die Kantenlänge ist {laenge} cm")
```

`print` ohne Zeilenumbuch

```
i=1
while(i<10):
```

```
    print(i, end=' ')
    i = i + 1


help(print)
```

Ausdrücke

- Alles, was sich zu einem Wert evaluieren lässt
- Konstanten, Variablen, Funktionsaufrufe
- Auch kombiniert mit Operatoren

```
4
a
a + 42
sum([1]) + 5
x = a + sum([1]) + 5 # Zuweisung
```

Boolesche Ausdrücke

- Lassen sich zu `True` oder `False` auswerten
- Werden für *Kontrollfluss* benötigt
- Vergleiche (>, >=, <, <=, ==, !=)
- Verknüpfungen (`and`,`or`,`not`)
- Werte, die als **False** gelten (u.a): `0`, `0.0`, `''`, `[]`, `None`

```
a = 1
x = a < 4
print(x == 1)

y = 0
print(x > 0 and y > 0)

li = ['a']
i = 1
print(i < len(li) and li[i] == 'b')
```

Kontrollfluss

Bedingte Ausführung

- Die Überprüfung einer *Bedingung* entscheidet, ob ein **Block** von Anweisungen ausgeführt wird
- Python verwendet für Blöcke keine Klammern (so wie viele andere Programmiersprachen) sondern **Einrückungen**

- Achtung: Bitte Leerzeichen verwenden (und **keine** Tabulatoren)!

```
x = 1
doit = "yes"
if x>0 and doit=="yes":
    x = 0
    res = 42
print(x)
```

Alternativen

```
a = 1 ; b = 1
if(a<b):
    print("Kleiner")
else:
    print("Groesser oder Gleich")
```

Alternativen

```
a = 1 ; b = 1
if(a<b):
    print("Kleiner")
elif(a>b):
    print("Groesser")
else:
    print("Gleich")
```

Ab Python 3.10 gibt es ein `match-case`-Konstrukt

```
radius = breite = hoehe = 2
#form = "Kreis"
form = input("Gib eine geometrische Figur ein: ")
match form:
    case "Kreis":
        print("Die Fläche ist", 3.14159 * radius * radius)
    case "Rechteck":
        print("Die Fläche ist", breite * hoehe)
    case "Dreieck":
        print("Die Fläche ist", 0.5 * breite * hoehe)
    case _:
        print("Unbekannte Form!")
```

Schleifen

- Python kennt 2 Arten von Schleifen
- `while`Solange eine Bedingung gilt, führe einen bestimmten Anweisungsblock aus
- `for`Für alle Elemente eine Folge, führe einen bestimmten Anweisungsblock aus
- Schleifen können *geschachtelt* sein

```
potenz = 1
while(potenz<1000):
    print(potenz)
    potenz *= 2


liste = [1, 'Zwei', 3.0]

print("Liste: ", end='')
for elem in liste:
    print(elem, end=' ')

print("\n\nrange: ", end='')
for i in range(10):
    print(i, end=' ')
```

Funktionen

- Der Name verweist auf das erzeugte Funktionsobjekt, das den Anweisungsblock enthält
- Namen sind Bezeichner und folgen den bekannten Syntaxregeln
- Funktionen können in Funktionen definiert werden
- Funktionen sind *First-Class Objects*: sie können dynamisch erzeugt, als Argumente übergeben oder zurückgegeben werden

```
def funktionsname(a, b=2, *args, **kwargs):
    """
    Kurze Beschreibung der Funktion.
    Parameter:
        a: Bedeutung von Parameter a
        b (optional): Bedeutung von parameter b
        *args: Weitere Positionsargumente
        **kwargs: Weitere benannte Argumente
    Rückgabe:
        Beschreibung des Rückgabewerts
    """
    print(f"a={a} b={b}", end=' ')
    for arg in args: print(arg, end=' ');
    for kwarg in kwargs: print(f"{kwarg}={kwargs[kwarg]}", end=' ');
    return (a,b,args,kwargs)
funktionsname(1, 3, 4, 5, c=6, d=7)


help(funktionsname)
```

Funktionsaufruf

- Funktionsnamen mit `()`
- Argumente übergeben
- Eine Funktion besitzt *Parameter*
- Beim Aufruf übergibt man *Argumente* an die Funktion
- Parameter können *default*-Werte besitzen (und sind damit optional)

```
def print_params(a,b,c):
    print(a)
    print(b)
    print(c)

print(1,2,3)
```

Aufrufsemantik

- Die Aufrufsemantik von Python ist Call-by-object-reference
- Die Parameter zeigen auf dieselben Objekte wie die Argumente
- Ändert sich auch der Wert eines Arguments (ausserhalb der Funktion), wenn man den Parameter innerhalb der Funktion ändert?
- Antwort: Das liegt daran, ob der Typ des Arguments ein *veränderbarer* Typ oder ein *nicht-veränderbarer* Typ ist

```
def foo(arg):
    arg += 1
    print(arg)


x = 1
foo(x)
print(x)
```

```
%%tutor --lang python3 --cumulative --heapPrimitives
def foo(arg):
    arg += [3,4]
    print(arg)

x = [1,2]
foo(x)
print(x)
```

Namensräume

Referenzen (Namen oder *Variablen*), denen innerhalb einer Funktion ein Wert zugewiesen wird, sind **lokal** für diese Funktion

- Existieren nur, solange Anweisungen dieser Funktion ausgeführt werden
- Nach dem Ende der Funktion sind die lokalen Namen nicht mehr **erreichbar** (lokale Referenzen gehen verloren)
- Mit den Schlüsselwörtern `global` oder `nonlocal` wird die Zuweisung an den globalen bzw. einen äußeren Namensraum gebunden

```
a = "globales a"
def f():
    a = "lokales a"
    print(a)

f()
print(a)
```

Rückgabewert

- Ein Rückgabewert ist der Wert, den eine Funktion nach ihrer Ausführung an den Aufrufer zurückgibt.
- Warum? Variablen, die nur innerhalb der Funktion definiert sind, gehen nach dem Funktionsaufruf verloren.
- Mit dem Schlüsselwort `return` wird ein Wert an die aufrufende Stelle zurückgegeben.

```
def my_pow(a,b):
    return a**b

zwei_hoch_acht = my_pow(2,8)
zwei_hoch_acht
```

Zeichenketten und Dateien

- Die wichtigste Methode für Strings ist:

```
help(str)
```

- **str ist ein nicht-veränderbarer, sequentieller Datentyp**

Notation	Beschreibung
`x in s`	Prüft, ob x in s enthalten ist. Das Ergebnis ist ein Wahrheitswert.
`x not in s`	Prüft, ob x nicht in s enthalten ist. Das Ergebnis ist eine bool-Instanz. Gleichwertig mit not x in s.
`s + t`	Das Ergebnis ist eine neue Sequenz, die die Verkettung von s und t enthält.

Notation	Beschreibung
`s += t`	Erzeugt die Verkettung von s und t und weist sie s zu.

Notation	Beschreibung
`s * n oder n * s`	Liefert eine neue Sequenz, die die Verkettung von n Kopien von s enthält.
`s *= n`	Erzeugt das Produkt s * n und weist es s zu.
`s[i]`	Liefert das i-te Element von s.
`s[i:j]`	Liefert den Ausschnitt aus s von i bis j.
`s[i:j:k]`	Liefert den Ausschnitt aus s von i bis j, wobei nur jedes k-te Element beachtet wird.

```
s = "Hallo Welt!"
s[6]
```

Notation	Beschreibung
`len(s)`	Gibt die Anzahl der Elemente von s zurück.
`max(s)`	Liefert das größte Element von s, sofern eine Ordnungsrelation für die Elemente definiert ist.
`min(s)`	Liefert das kleinste Element von s, sofern eine Ordnungsrelation für die Elemente definiert ist.
`s.index(x)`	Gibt den Index des ersten Vorkommens von x in der Sequenz s zurück.
`s.count(x)`	Zählt, wie oft x in der Sequenz s vorkommt.

- Textersetzung mit `replace`
- Umwandlung Groß-/Kleinschreibung mit `lower` und `upper`

```
'Übergröße'.lower().replace('ü','ue').\
    replace('ö','oe').replace('ß','ss').upper()
```

- Mehrere Ersetzungen gleichzeitig mit `translate` und `makerans`

```
de_table = str.maketrans({'Ä': 'Ae', 'Ö': 'Oe', 'Ü': 'Ue',
                          'ä': 'ae', 'ö': 'oe', 'ü': 'ue', 'ß': 'ss'})
'Übergröße'.translate(de_table)
```

Dateien

- Öffnen von Datein mit `open` im Modus *read* (`r`), *write* (`w`) oder *append* (`a`)
- Schließen mit `file.close()`
- `file.read()` ließt den gesamten Inhalt von `file`
- Zeilenweises Durchlaufen mit `for`
- Zugriff auf Dateien kann fehlschlagen. Daher immer `try-catch` oder *Context Manager* (`with`-Statement) verwenden

```
try:
    f = open('faust-utf8.txt','r')
    i = 1
    for line in f:
        print(f"{i:6d}: ",line,end='')
        if i==20:
            break
        i += 1
    f.close()
except:
    print('Datei konnte nicht geoeffnet werden')
```

```
f = open('faust-utf8.txt','r')
f_neu = open("absatz.txt", 'w')
i = 1
for line in f:
    if i>=1359 and i<=1361:
        f_neu.write(line)
    i += 1
    if i>1361: break;
f.close()
f_neu.close()
```

```
f = open('absatz.txt','r')
for line in f:
    print(line.replace('ü','ue'), end='')
f.close()
```

Zum 3. Termin

- Listen, Tupel und Dictionaries
- Wenn wir dort sind, können wir viele sinnvolle Programme schreiben:-)